

```
In Base: a_ = 10
In Derived: a_ = 11 and b_ = 21
In Derived: a_ = 11 and b_ = 21
In Derived: a_ = 15 and b_ = 21<-- there was never an object
with (15, 21)!
```

What essentially has happened is that the object reference *bb* has some portion of object *d* and some portion of object *dd*. This happened because *bb = dd* made the derived part get sliced and therefore *bb* never got the value *b_ = 25*. So it retained the old value of *b_* as 21 (from the initialisation) and used the new value of *a_* as 15.

As can be seen, this subtle effect of object slicing can throw your program and understanding completely off course.

How to prevent it

A very simple but harsh solution is to make the base class abstract and disallow object creation. But this solution might not be applicable in many instances where one has a need to instantiate the base class or when one has multi-level inheritance needs.

An alternate solution

Here one needs to declare assignment operator as virtual in the base class. This way the invocation will know which exact operator (base or derived) to call at runtime. You also need to define the overridden assignment operator in the derived class to accept the base class object as the parameter. After all, that's what you intend to do (in the troublesome assignment mentioned above). We then need to define a *copy()* method, which will copy the members appropriately. The derived class's assignment operator needs to downcast the given base object and call these *copy()* methods appropriately.

Remember, this approach has to be implemented for all derived classes. You can also choose to define the *copy()* method as protected if you want to avoid direct invocation of it.

The following complete example shows how this works.

```
#include <iostream>
using namespace std;

class Base
{
public:
    Base(int a)
    {
        a_ = a;
    }

    virtual Base & operator = (const Base &ob)
    {
        copy (ob);
        return *this;
    }
};
```

```
    }

    virtual void print()
    {
        cout<< "In Base: a_ = " << a_ <<endl;
    }

    void copy (const Base &other)
    {
        this->a_ = other.a_;
    }

protected:
    int a_;
};

class Derived : public Base
{
public:
    Derived(int a, int b):Base(a)
    {
        b_ = b;
    }

    virtual Derived & operator = (const Base &base)
    {
        if (const Derived *der = dynamic_cast<const Derived *>
            (&base))
        {
            copy (*der);    // calling Derived::copy()
            return *this;
        }
        else
        { // the cast could not succeed.
            cerr<< "Error! Wrong cast invoked.";
            exit (1);
        }
    }

    void print()
    {
        cout<< "In Derived: a_ = " << a_ << " and b_ = " << b_
            <<endl;
    }

    void copy (const Derived &other)
    {
        Base::copy(other);    // Let Base::copy() handle
        copying Base things
        this->b_ = other.b_;
    }

private:
```

To be continued on page 45...